

6.1 Analiza certyfikatu SSL/TLS witryny.

- (a) Za pomocą narzędzia **OpenSSL**, pobierz certyfikat strony internetowej <https://github.com>.

```
echo | openssl s_client -servername github.com -connect github.com:443 2>/dev/null | openssl x509 -out github.crt
```

- (b) Wyświetl certyfikat w czytelnej postaci.
- (c) Przeanalizuj certyfikat i odczytaj z niego następujące informacje (używając narzędzia **OpenSSL**):
- Dla jakiej domeny wystawiony jest certyfikat?
 - Kto wystawił certyfikat (CA)?
 - Kiedy certyfikat został wystawiony i kiedy wygasa?
 - Jakie dodatkowe domeny są obsługiwane?
 - Jaki algorytm i długość klucza?
 - Do czego może być użyty certyfikat?
 - Ile certyfikatów znajduje się w łańcuchu zaufania?
 - Kto jest głównym CA (root CA) w łańcuchu?
- (d) Za pomocą narzędzia **OpenSSL**, wyodrębnij z certyfikatu klucz publiczny i zapisz go do pliku.


```
(.python_env) [ ]-[admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -text -noout  
Certificate:  
  Data:  
    Version: 3 (0x2)  
    Serial Number:  
      ab:66:86:b5:62:7b:e8:05:96:82:13:30:12:86:49:f5  
    Signature Algorithm: ecdsa-with-SHA256  
    Issuer: C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo ECC Domain Validation Secure Server CA  
    Validity  
      Not Before: Feb  5 00:00:00 2025 GMT  
      Not After : Feb  5 23:59:59 2026 GMT  
    Subject: CN = github.com  
    Subject Public Key Info:  
      Public Key Algorithm: id-ecPublicKey  
      Public-Key: (256 bit)  
      pub:  
        04:20:34:5c:46:ff:2c:cb:f8:24:9a:ae:f0:bb:2f:  
        77:a9:1f:97:21:36:71:ba:c2:26:18:c5:1e:43:fd:  
        9d:49:e0:cc:46:9c:85:fc:29:b4:f9:7c:28:0b:a3:  
        2c:c7:5c:bf:6f:e7:46:dd:04:8a:ba:cb:80:2d:37:  
        88:0d:ee:06:d6  
      ASN1 OID: prime256v1  
      NIST CURVE: P-256  
  X509v3 extensions:  
    X509v3 Authority Key Identifier:  
      F6:85:0A:3B:11:86:E1:04:7D:0E:AA:0B:2C:D2:EE:CC:64:7B:7B:AE  
    X509v3 Subject Key Identifier:  
      53:C8:7F:DE:9E:98:4E:C7:4D:D6:BC:DE:AB:95:3E:30:3D:3D:D1:C8  
    X509v3 Key Usage: critical  
      Digital Signature  
    X509v3 Basic Constraints: critical  
      CA:FALSE  
    X509v3 Extended Key Usage:  
      TLS Web Server Authentication, TLS Web Client Authentication  
    X509v3 Certificate Policies:  
      Policy: 1.3.6.1.4.1.6449.1.2.2.7
```

CPS: https://sectigo.com/CPS
Policy: 2.23.140.1.2.1
Authority Information Access:
CA Issuers - URI:http://crt.sectigo.com/SectigoECCDomainValidationSecureServer
CA.crt

OCSP - URI:http://ocsp.sectigo.com
CT Precertificate SCTs:
Signed Certificate Timestamp:
Version : v1 (0x0)
Log ID : 96:97:64:BF:55:58:97:AD:F7:43:87:68:37:08:42:77:
E9:F0:3A:D5:F6:A4:F3:36:6E:46:A4:3F:0F:CA:A9:C6
Timestamp : Feb 5 00:03:50.475 2025 GMT
Extensions: none
Signature : ecdsa-with-SHA256
30:44:02:20:3B:8B:AA:3E:2E:94:23:B7:A0:1E:12:39:
6D:1E:1B:3F:4E:21:02:9C:77:74:C5:37:9E:AF:FD:EF:
0F:5C:60:B0:02:20:62:51:B0:46:8B:7E:4A:A1:01:0A:
CF:FF:7E:BC:7F:60:74:CF:C2:8C:7D:40:B7:72:EC:68:
D3:2D:61:DF:70:C0
Signed Certificate Timestamp:
Version : v1 (0x0)
Log ID : 19:86:D4:C7:28:AA:6F:FE:BA:03:6F:78:2A:4D:01:91:
AA:CE:2D:72:31:0F:AE:CE:5D:70:41:2D:25:4C:C7:D4
Timestamp : Feb 5 00:03:50.381 2025 GMT
Extensions: none
Signature : ecdsa-with-SHA256
30:46:02:21:00:E5:AC:89:F1:EC:9F:B7:31:FA:A0:C4:
1D:BE:16:FA:B7:74:C9:64:D6:A8:F8:72:13:B0:FF:E1:
E4:61:57:C6:D0:02:21:00:C4:FE:24:A0:10:7D:4B:88:
74:11:B1:7E:BC:AB:1A:BC:2C:38:3D:E9:46:CD:6D:C8:
0C:B2:91:D3:C6:46:0B:13
Signed Certificate Timestamp:
Version : v1 (0x0)
Log ID : CB:38:F7:15:89:7C:84:A1:44:5F:5B:C1:DD:FB:C9:6E:
F2:9A:59:CD:47:0A:69:05:85:B0:CB:14:C3:14:58:E7
Timestamp : Feb 5 00:03:50.437 2025 GMT
Extensions: none
Signature : ecdsa-with-SHA256

30:45:02:21:00:D4:62:96:F7:66:A0:0C:53:49:21:8A:
CC:1F:78:1A:25:D5:EC:74:85:69:51:C6:4E:7F:11:F5:
16:4B:1B:B8:B1:02:20:52:42:7E:C9:48:36:17:39:DD:
0D:13:20:C2:47:75:C1:4E:5B:6B:60:1B:8B:41:03:57:
4B:F3:CD:6D:5D:B3:27
X509v3 Subject Alternative Name:
DNS:github.com, DNS:www.github.com
Signature Algorithm: ecdsa-with-SHA256
Signature Value:
30:44:02:20:71:8c:a7:6e:c1:04:12:75:df:9e:a5:09:ed:96:
63:2c:d8:22:9f:df:00:e3:50:33:70:24:78:4f:df:ca:6d:2c:
02:20:6d:55:f3:77:62:02:19:fa:77:87:11:fc:1c:46:18:73:
e2:e0:e9:73:c1:7e:b4:a9:ad:71:e5:89:4a:27:0c:90

C)

2)

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -noout -subject  
subject=CN = github.com
```

3)

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -noout -issuer  
issuer=C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo ECC Domain Validation Secure Server CA
```

4)

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -noout -dates  
notBefore=Feb  5 00:00:00 2025 GMT  
notAfter=Feb  5 23:59:59 2026 GMT
```

5)

```
└─$ openssl x509 -in github.crt -noout -fingerprint -sha256  
sha256 Fingerprint=B8:BB:81:87:68:33:87:39:42:04:5A:8D:F8:F0:62:19:E0:06:02:EB:CB:43:84:C7:AB:C2:4F:18:37:9C:87:F5
```

6)

```
└─$ openssl x509 -in github.crt -noout -ext subjectAltName  
X509v3 Subject Alternative Name:  
    DNS:github.com, DNS:www.github.com
```

7)

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -noout -keyUsage  
(.python_env) [admin@parrot]-[~]  
└─$
```

Jakie były algorytmy zastosowane)

```
(.python_env) [x]-[admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -text -noout | grep "Public Key Algorithm"  
Public Key Algorithm: id-ecPublicKey
```

ilo bitowe były klucze)

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl x509 -in github.crt -text -noout | grep "Public-Key"  
Public-Key: (256 bit)
```

d)

```
(.python_env) [admin@parrot]~  
└─$ openssl x509 -in github.crt -noout -pubkey > github.pubkey  
(.python_env) [admin@parrot]~  
└─$ cat github.pubkey  
-----BEGIN PUBLIC KEY-----  
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEIDRcRv8sy/gkmaq7wuy93qR+XITZx  
usImGMUeQ/2dSeDMRpyF/Cm0+XwoC6Msx1y/b+dG3QSKusuALTeIDe4G1g==  
-----END PUBLIC KEY-----
```

- 6.2** Pobierz cały łańcuch certyfikatów z serwera <https://github.com> przy użyciu OpenSSL i sprawdź liczbę certyfikatów w łańcuchu. Następnie wskaż, który certyfikat należy do serwera, a które do CA. Użyj poniższego polecenia:

```
openssl s_client -connect github.com:443 -showcerts </dev/null > gh.txt 2>/dev/null
```

```
(.python_env) [admin@parrot]~[~]
└─$ openssl s_client -connect github.com:443 -showcerts </dev/null > gh.txt 2>/dev/null
(.python_env) [admin@parrot]~[~]
└─$ cat gh.txt
CONNECTED(00000003)
---
Certificate chain
 0 s:CN = github.com
   i:C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo ECC Domain Validation Secure Server CA
   a:PKKEY: id-ecPublicKey, 256 (bit); sigalg: ecdsa-with-SHA256
   v:NotBefore: Feb  5 00:00:00 2025 GMT; NotAfter: Feb  5 23:59:59 2026 GMT
-----BEGIN CERTIFICATE-----
MIIEOwTCCBEigAwIBAgIRAKtmhrVie+gFloITMBKGSfUwCgYIKoZIzj0EAwIwY8x
CzAJBgNVBAYTAkdCMRswGQYDVQQIEiJHcmVhdGVyIE1hbmNoZXN0ZXIxEDA0BgNV
BACTB1NhbGZvcmlkXGQDAWBgNVBAoTD1NlY3RpZ28gTGltZXN0ZXI3MDUGA1UEAxMu
U2VjdGlnbyBFQ0MGRG9tYVluIFZhbG1kYXRpb24gU2VjdXJlIFNlcnZlcjB0QTAe
Fw0yNTAyMDUwMDAwMDAwFw0yNTAyMDUyMzU5NTl1aMBUxEzARBGNVBAMTCmdpdGh1
Yi5jb20wWTATBgqhkiJOPQIBBggqhkiJOPQMBBwNCAAQgNFxG/yzL+CSarvc7L3ep
H5chNnG6wiYYxR5D/Z1J4MxGnIX8KbT5fCgLozHXL9v50bdBIq6y4AtN4gn7gbw
o4IC/DCCAvGwHwYDVR0jBBgwFoAU9oUK0xGG4QR9DqoLLNLuzGR7e64wHQYDVR00
BBYEFfPiF96emE7HTda83quVPjA9PdHIMA4GA1UdDwEB/wQEAwIHgDAMBGNVHRMB
Af8EAjAAMB0GA1UdJQQwMBQGCCsGAQUFBwMBBggrBgEFBQcDAjBjBGNVHSAEQjBA
MDQCysGAQQBsjEBAGIHMCIUwIwYIKwYBBQUHAQEWF2h0dHBzO18vc2VjdGlnby5j
b20vQ1B1MTAGBgBmeBDAEATCBhAYIKwYBBQUHAQEEdB2ME8GCCsGAQUFBzAChkNo
dHRwO18vY3J0LnNlY3RpZ28uY29tL1NlY3RpZ29FQ0NEb21hZW5WYWxpZGF0aW9u
U2VjdXJlU2VydmlVY0EuY3J0MCMGCCsGAQUFBzABhhodHRwO18vb2NzcC5zZWNO
aWdvLmNvbTCCAX4GCIsgAQQB1nkCBAIEggFuBIIBagFoAHUAlpdkv1VY1633Q4do
NwhCd+nwOtX2pPM2bkakPw/KqcYAAAGU02uUSwAABAMARjBEAiA7i6o+LpQjt6AE
EjltHhs/TiECnHd0xTeer/3vD1xgsAIgYlGwRot+SqEBCs//frx/YHTPwox9QLdy
7GjTLWHfcMAADwAZhtTHKKpv/roDb3ggTQGRqs4tcjEPrs5dcEEtJUzH1AAAAZTT
a5PtAAAAEAwBIMEYCIQD1rInx7J+3MfqqxB2+Fvq3dMlk1qj4ch0w/+HkYVfG0AIh
AMT+JKAQfUuIdBGxfryrGrwsOD3pRs1tyAyykdPGRgsTAHYAyzj3FY18hKFEX1vB
3fvJbvKaWc1HCmkFhbDLFMMUWocAAAGU02uUJQAABAMARzBFAiEA1GKw92agDFNJ
IYrMH3gaJdXsdIVpUcZ0fxH1FksbuLECIFJCfs1INhc53Q0TIMJHdcFOW2tgg4tB
A1dL881tXbMnMCUGA1UdEQQeMByCCmdpdGh1Yi5jb22CDnd3dy5naXRodWIUy29t
MAoGCCqGSM49BAMCA0cAMEQCIHGMp27BBBj13561Ce2WYyzYIp/faONQM3AkeE/f
ym0sAiBtVfN3YgIZ+neHEfwCRhhz4uDpc8F+tKmtceWJSicMkA==
```


MIID0zCCArugAwIBAgIQVmcdB0pPmUxvEIFHwdJ1lDANBgkqhkiG9w0BAQwFADB7
MQswCQYDVQQGEwJHqjEbmBKA1UECAwSR3JlYXRlcjBNYW5jaGVzdGVyMRAwDgYD


```
VQQHDA dTYWxmb3JkMRowGAYDVQKDBFDb21vZG8gQ0EgTG1taXRlZDEhMB8GA1UE
AwwYQUFBIEIENlcnRpZmljYXRlIFNlcnZpY2VzMB4XDTE5MDMxMjAwMDAwMFoXDTE4
MTIzMjIzNTk1OVowGyGxCzAJBgNVBAYTA1VTMRMwEQYDVQIEwpoZXXcgSmVyc2V5
MRQwEgYDVQKHEwtKZXJzZXkgQ2l0eTEeMBwGA1UEChMVVGhlIFVTRVJUUVlVTVCB0
ZXR3b3JrMS4wLAYDVQDEyVU0VSUHJ1c3QgRUNDIENlcnRpZmljYXRpb24gQXV0
aG9yaXR5MHHYwEAYHKoZIzj0CAQYFK4EEACIDYgAEGqxUWqn5aCPnetUkb1PGWthL
q8bVttHmc3Gu3ZzWDGH926CJA7gFF0xXzu5dP+Ihs8731Ip54KODfi2X0GHE8Znc
JZFjq38wo7Rw4sehM5zzvy5cU7Ffs30yf4o04315o4HyMIHvMB8GA1UdIwQYMBaA
FKARCiM+lvEH7OKvKe+CpX/QMKs0MB0GA1UdDgQWBBQ64QmG1M8ZwpZ2dEl23OA1
xmNjMjA0BgNVHQ8BAf8EBAMCAYYwDwYDVR0TAQH/BAUwAwEB/zARBgNVHSAECjAI
MAYGBFUdIAAwQwYDVR0fBDwwOjA4oDagNIYyaHR0cDovL2Nybc5jb21vZG9jYS5j
b20vQUFBQ2VydGlmawNhdGVtZXJ2aWNlcy5jcmmwNAYIKwYBBQUHAQEEDAMMCQG
CCsGAQUFBzABhhodHRwOi8vb2NzcC5jb21vZG9jYS5jb20wDQYJKoZIhvcNAQEM
BQADggEBABns652JLCALBIAdGN5CmXKZFjK9Dpx1WywV4i1Abe7/ctvbq5AfjJXy
ij0IckKUAFiORVsAYfZFhr1wHUrxeZWEQff2Ji8fJ8Z0d+LygBkc7xGEJuTI42+
FsMuCIKchjN0djs0TI0DQoWz4rIjQtUfenVqGtF8qmchxDM60W1TyaLtYiKou+JV
bJLsQ2uRl9EMC5MCHdK8aXdJ5htN978UeA0wproLtOGFfy/cQjutdAFI3tZs4RmY
CV4Ks2dH/hzg1cEo70qLRDEmBDeNiXQ2Lu+lIg+DdEmSx/cQwgwp+7e9un/jX9Wf
8qn0dNW44b0wgeThpW0jz0oEeJBuv/c=
```

-----END CERTIFICATE-----

Server certificate

subject=CN = github.com

issuer=C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo ECC Domain Validation Secure Server CA

No client certificate CA names sent

Peer signing digest: SHA256

Peer signature type: ECDSA

Server Temp Key: X25519, 253 bits

SSL handshake has read 3480 bytes and written 380 bytes

Verification: OK

New, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256

Server public key is 256 bit

Secure Renegotiation IS NOT supported

Compression: NONE

Expansion: NONE

No ALPN negotiated

Early data was not sent

Verify return code: 0 (ok)

6.3 Analiza certyfikatu SSL/TLS lokalnego serwera (certyfikatu typu self-signed).

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8443:443 --name tls3 docker.io/mazurkatarzyna/tls-ex3:latest  
podman run -p 8443:443 --name tls3 docker.io/mazurkatarzyna/tls-ex3:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8443:443 --name tls3 ghcr.io/mazurkatarzynaumcs/tls-ex3:latest  
podman run -p 8443:443 --name tls3 ghcr.io/mazurkatarzynaumcs/tls-ex3:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8443>.

- (b) Wyświetl certyfikat w czytelnej postaci.
- (c) Przeanalizuj certyfikat i odczytaj z niego następujące informacje (używając narzędzia **OpenSSL**):
- Dla jakiej domeny wystawiony jest certyfikat?
 - Kto wystawił certyfikat (CA)?
 - Kiedy certyfikat został wystawiony i kiedy wygasa?
 - Jakie dodatkowe domeny są obsługiwane?
 - Jaki algorytm i długość klucza?

Bezpieczeństwo Systemów Komputerowych - Transport Layer Security (TLS) | Katarzyna Mazur

- Do czego może być użyty certyfikat?
- Ile certyfikatów znajduje się w łańcuchu zaufania?
- Kto jest głównym CA (root CA) w łańcuchu?

- (d) Za pomocą narzędzia **OpenSSL**, wyodrębnij z certyfikatu klucz publiczny i zapisz go do pliku.

b)

```
(.python_env) [admin@parrot]~  
└─$ echo | openssl s_client -servername localhost -connect 127.0.0.1:8443 2>/dev/null | open  
ssl x509 -out local1.crt  
(.python_env) [admin@parrot]~  
└─$ openssl x509 -in local1.crt -text -noout  
Certificate:  
  Data:  
    Version: 3 (0x2)  
    Serial Number:  
      1f:d6:f4:ca:fd:73:32:e4:1e:37:e7:ea:3a:12:72:aa:36:73:27:98  
    Signature Algorithm: sha256WithRSAEncryption  
    Issuer: C = PL, ST = Lubelskie, L = Lublin, O = UMCS, OU = MFI, CN = localhost  
    Validity  
      Not Before: Dec  4 17:29:18 2025 GMT  
      Not After : Dec  4 17:29:18 2026 GMT  
    Subject: C = PL, ST = Lubelskie, L = Lublin, O = UMCS, OU = MFI, CN = localhost  
    Subject Public Key Info:  
      Public Key Algorithm: rsaEncryption  
      Public-Key: (2048 bit)  
      Modulus:  
        00:d7:5a:e1:4a:24:fd:e1:7e:bb:8d:74:f9:3e:0f:  
        48:2b:21:af:63:23:73:d5:9c:68:bf:20:4f:c8:44:  
        94:85:a5:08:3d:2b:ee:23:b8:f4:64:05:fa:96:0c:  
        19:4d:68:00:be:e3:42:15:61:f5:fc:be:fa:46:8a:  
        69:77:f0:30:23:3e:04:d6:6a:41:1e:81:24:7b:0a:  
        dd:b5:4c:22:77:a7:a0:d6:82:a3:6e:d8:c5:a7:7c:  
        29:5b:03:f3:cf:e9:70:1f:08:64:5f:82:b4:39:b5:  
        3e:bc:bc:ae:46:39:fb:b3:aa:36:3b:76:84:03:d1:  
        3b:b3:ca:9b:fb:31:39:66:20:da:ee:47:04:92:69:  
        6e:c7:9f:8b:9c:88:ae:56:02:ba:00:76:20:49:c9:  
        54:bf:0e:9a:e1:00:ba:6f:c1:b4:0a:e2:b7:81:d8:  
        ab:b0:d7:de:ac:50:f3:b0:58:97:de:39:41:71:a6:  
        69:ec:b8:c8:0c:25:14:ff:b9:a2:e4:db:44:c6:42:  
        73:59:ee:64:bb:86:45:55:ea:a9:d5:a5:ea:7e:ed:  
        3a:d0:6e:46:18:97:b9:b1:0f:f5:f7:5e:e8:8e:28:  
        ec:8d:98:2a:20:2f:28:aa:1f:fc:ae:21:14:ca:9a:  
        c2:33:f0:ae:f0:a7:8f:30:79:79:0a:3c:9d:57:32:  
        fe:79
```

```

    Exponent: 65537 (0x10001)
X509v3 extensions:
    X509v3 Subject Key Identifier:
        2B:79:0E:09:63:FB:C6:5A:2F:84:6F:3B:F1:6B:6A:5D:1E:AE:C9:C6
    X509v3 Authority Key Identifier:
        2B:79:0E:09:63:FB:C6:5A:2F:84:6F:3B:F1:6B:6A:5D:1E:AE:C9:C6
    X509v3 Basic Constraints: critical
        CA:TRUE
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
    cf:28:ad:83:cf:d6:8c:30:95:a7:8d:47:6f:84:8e:d0:5b:45:
    bf:a4:7b:2a:bd:67:b6:81:ed:22:e5:cc:78:30:5f:af:ef:fd:
    19:bb:b3:3b:c8:f4:b6:90:17:8b:04:b6:c9:ce:ee:27:6c:cc:
    a0:b9:ec:2e:6e:78:5d:2f:8c:c6:cf:f6:59:a9:b6:af:30:14:
    24:9a:16:55:a8:44:04:8f:35:e8:0a:a4:ad:a7:d0:0e:73:24:
    05:fa:4b:21:e1:ac:7c:79:b1:50:99:cd:e0:a4:f3:d3:b8:0e:
    7c:6c:c7:1d:2c:b2:27:4a:9a:d6:15:9e:43:75:bc:56:d3:60:
    2d:a4:db:01:0f:c8:a8:d8:11:7c:14:40:0a:52:66:ef:64:a3:
    d8:8c:28:8d:1d:d9:11:76:63:a4:5f:c2:83:f0:7e:19:07:84:
    61:3c:e7:f2:9a:b5:6b:34:2c:e0:0e:57:b4:35:34:5d:06:d5:
    ac:d9:73:61:69:4d:fb:20:84:15:c4:c7:11:92:74:57:4f:45:
    04:cb:26:db:dd:7c:94:d0:dd:72:69:12:d7:07:be:68:2a:3e:
    35:84:8f:17:0d:29:59:c7:b3:c3:ce:58:44:c6:1e:72:64:a7:
    17:1c:dc:51:5c:c3:9d:29:bc:59:a5:81:0b:bb:f0:4a:08:30:
    b7:fd:0d:cb

```

c)

```

(.python_env) [admin@parrot]~$
└─$ openssl x509 -in local1.crt -noout -issuer
issuer=C = PL, ST = Lubelskie, L = Lublin, O = UMCS, OU = MFI, CN = localhost
(.python_env) [admin@parrot]~$
└─$ openssl x509 -in local1.crt -noout -subject
subject=C = PL, ST = Lubelskie, L = Lublin, O = UMCS, OU = MFI, CN = localhost
(.python_env) [admin@parrot]~$

```

6.5 Pod adresem <https://127.0.0.1:8447> działa prosty serwer pozwalający przetestować swoją wiedzę odnośnie certyfikatów TLS. Serwer posiada 2 endpointy, <https://127.0.0.1:8447/cert>, pod którym zwraca losowy certyfikat, oraz <https://127.0.0.1:8447/validate>, pod który można wysłać swoje odpowiedzi, i sprawdzić, czy prawidłowo przeanalizowaliśmy certyfikat.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8447:443 --name tls6 docker.io/mazurkatarzyna/tls-ex6:latest
podman run -p 8447:443 --name tls6 docker.io/mazurkatarzyna/tls-ex6:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8447:443 --name tls6 ghcr.io/mazurkatarzynaumcs/tls-ex6:latest
podman run -p 8447:443 --name tls6 ghcr.io/mazurkatarzynaumcs/tls-ex6:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8447>.

- (b) Wyślij request do endpointa <http://127.0.0.1:8447/cert> używając metody HTTP GET. Otrzymasz w odpowiedzi certyfikat TLS - zapisz go do pliku o nazwie `cert.pem`.
- (c) Z pobranego certyfikatu wyodrębnij klucz publiczny. Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:8447/public> wygenerowany klucz publiczny (jako plik, `public_key`). W odpowiedzi serwer zwróci informację, czy klucz pasuje do certyfikatu.
- (d) Wyświetl certyfikat (pobrany z serwera plik `cert.pem`) w czytelnej postaci, przeanalizuj go i odpowiedz na poniższe pytania:
- Kiedy wygasa certyfikat (data)? (`expiry_date`)
 - Kiedy certyfikat został wydany (data)? (`issued_date`)
 - Kto jest wydawcą certyfikatu (Root CA)? (`issuer`)
 - Jaki algorytm został wykorzystany do wygenerowania klucza? (`key_algorithm`)
 - Jaki jest rozmiar klucza? (`key_size`)
 - Jaka jest główna domena certyfikatu? (`primary_domain`) Jakie są alternatywne domeny? (`san_domains`)
 - Do czego może być użyty klucz? (`key_usage`)
- (e) Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:8447/validate> swoje odpowiedzi na powyższe pytania. Serwer przyzna punkty za każdą poprawną odpowiedź. (Maksymalnie można zdobyć 6 punktów).

Podpowiedź: Możesz skorzystać z endpointa <http://127.0.0.1:8447/apidocs>, aby sprawdzić, jak przesyłać swoje odpowiedzi do serwera.

```
(.python_env) └─[admin@parrot]-[~]
└─ $curl -s -o cert.pem -X GET http://localhost:8447/cert
(.python_env) └─[admin@parrot]-[~]
└─ $openssl x509 -in cert.pem -noout -pubkey > cert.pubkey
(.python_env) └─[admin@parrot]-[~]
└─ $curl -X POST "http://127.0.0.1:8447/public" -H "accept: application/json" -H "Content-Type: multipart/form-data" -F "public_key=@cert.pubkey"
{
  "certificate_info": "Domain: api594.pl, Issuer: GlobalSign Authority",
  "matches": true,
  "message": "Klucz publiczny zgadza się z certyfikatem!"
}
(.python_env) └─[admin@parrot]-[~]
└─ $cat cert.pem
-----BEGIN CERTIFICATE-----
MIIDTjCCAjaGAWIBAgIU1t4c8lhPCFZUnqLTw6RFussyWswDQYJKoZIhvcNAQEL
BQAwQTEdMBsGA1UEAwUR2xvYmFsU2lnbiBBdXRob3JpdHkxEzARBgNVBAoMCKds
b2JhbFNPZ224xCzAJBgNVBAYTAlBMMB4XDTE1MTIwODEwMDU1M1oXDTI2MDIxOTEw
MDU1M1owMzESMBAGA1UEAwYJYXBpNTk0LnBsMRAwDgYDVQQKDAdPcmcgNzg2MQsw
CQYDVQQGEwJQTECCAS1wDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBALYzQk3s
6sa2M6QVqMrNH6kHQZGWr43Wz+NrPEzYb2sLbyhCdDfTvmSZLEza1rwED6xC4tZG
6WJP+ku1nmt1RxPJMsthEb+logBsW7Qy4ddtjWEoCVm+x0jN2zyt3uVOKy72jpi0
pMha0WQzVYjgB8Lqn3L2SS5b/wZOX/e0jJkFwEgXgWLMrTi0rezK6DyG5CaFsKvK
siFcPC6y09adHR+hfziqd/MhNmpXDVU+GfZqTHvP51371AY9pFj2NjKsoSJ4qCjD
rfZzmipYRQXB6WAV1i1PCK7+1075jQhPRMx1YcpOUAKOac0i5W8vkrGaf0W9zba0
aaCKDh0zAsJD7ysCAwEAAaNMMEowIAYDVR0RBbkWf4IjYXBpNTk0LnBsGgp3ZWIX
MTMuZWRI1MA4GA1UdDwEB/wQEAwIFoDAWBgNVHSUBAf8EDDAKBgggBgEFBQcDATAN
BgkqhkiG9w0BAQsFAAOCAQEAPqz0l3jkxb1/Hdmrd+5oCNumaygDZO/k5savbPHk
o8mAgixh/quH/8BG1vd11Pih2ftK4L4tvYDQgXTGjy5lIakxk9ck209EFUsbAfNA
z9s7X19NYL/VhAdMVvjUsKT6Yo433GP/r16er3lnr7g3Defd9n5Dq5s90hzJ4KH3
a0EL/MtyQ+3KIntJTDM18DU4jqRdxwfMsdeq0LVGAa56Q07SFBjYlm0/OvN1updq
uJXihuy+ZqDpJD+G4IXSKjRZfoqZFyf8g8Sg+dJ360L9Fyi/DZvCT8D2Nm+GR7bi
ycMxLWpQog8Pix2TTjr0xmtczEhQLk/2P8J4N+VaD7Kglw==
-----END CERTIFICATE-----
(.python_env) └─[admin@parrot]-[~]
```



```
$ openssl x509 -in cert.pem -text -noout
```

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

2f:5b:78:73:c9:61:3c:21:59:52:7a:8b:4f:0e:91:16:eb:2c:c9:6b

Signature Algorithm: sha256WithRSAEncryption

Issuer: CN = GlobalSign Authority, O = GlobalSign, C = PL

Validity

Not Before: Dec 8 10:05:53 2025 GMT

Not After : Feb 19 10:05:53 2026 GMT

Subject: CN = api594.pl, O = Org 786, C = PL

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:b6:33:42:4d:ec:ea:c6:b6:33:a4:15:a8:ca:cd:
1f:a9:07:41:91:96:af:8d:d6:cf:e3:6b:3c:4c:d8:
6f:6b:0b:6f:28:42:74:37:d3:be:64:99:2c:4c:da:
d6:bc:04:0f:ac:42:e2:d6:46:e9:62:4f:fa:4b:b5:
9e:6b:75:47:13:c9:32:cb:61:11:bf:b5:a2:00:6c:
5b:b4:32:e1:d7:6d:8d:61:28:09:59:be:c4:e8:cd:
db:3c:ad:de:e5:4e:2b:2e:f6:8e:98:b4:a4:c8:5a:
d1:64:33:55:88:e0:07:c2:ea:9f:72:f6:49:2e:5b:
ff:06:4e:5f:f7:8e:8c:99:05:c0:48:17:81:62:cc:
ad:38:b4:ad:ec:ca:e8:3c:86:e4:26:85:b0:ab:ca:
b2:21:5c:3c:2e:b2:d3:d6:9d:1d:1f:a1:7f:38:aa:
77:f3:21:36:6a:57:0d:55:3e:19:f6:6a:4c:7b:cf:
e7:5d:fb:d4:06:3d:a4:58:f6:36:32:ac:a1:22:78:
a8:28:c3:ad:f6:73:9a:2a:58:45:05:c1:e9:60:15:
d6:2d:4f:08:ae:fe:d7:4e:f9:8d:08:4f:44:cc:75:
61:ca:4e:50:02:8e:69:cd:22:e5:6f:2f:92:b8:00:
7f:45:bd:cd:b6:8e:69:a0:8a:0e:13:b3:02:c2:43:
ef:2b

```
Exponent: 65537 (0x10001)
X509v3 extensions:
  X509v3 Subject Alternative Name:
    DNS:api594.pl, DNS:web113.edu
  X509v3 Key Usage: critical
    Digital Signature, Key Encipherment
  X509v3 Extended Key Usage: critical
    TLS Web Server Authentication
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
  3e:ac:f4:97:78:e4:c5:bd:7f:1d:d9:ab:77:ee:68:08:db:a6:
  6b:28:03:64:ef:e4:e6:c6:af:6c:f1:e4:a3:c9:80:82:2c:61:
  fe:ab:87:ff:c0:46:d6:f7:75:d4:f8:a1:d9:fb:4a:e0:be:2d:
  bd:80:d0:81:74:c6:8f:2e:65:21:a9:31:93:d7:24:d8:ef:44:
  15:4b:1b:01:f3:40:cf:db:3b:5f:5f:4d:60:bf:d5:84:07:4c:
  56:f8:d4:b0:a4:fa:62:8e:37:dc:63:ff:ae:5e:9e:af:79:67:
  af:b8:37:0d:e7:dd:f6:7e:43:ab:9b:3d:d2:1c:c9:e0:a1:f7:
  68:e1:0b:fc:cb:72:43:ed:ca:20:db:49:4c:33:25:f0:35:38:
  8e:a4:5d:c7:07:cc:b1:d7:aa:d0:b5:46:01:ae:7a:40:ee:d2:
  14:12:72:96:63:bf:3a:f3:75:ba:97:6a:b8:95:e2:86:ec:be:
  66:a0:e9:24:3f:86:e0:85:d2:2a:34:59:7e:8a:99:17:27:fc:
  83:c4:a0:f9:d2:77:eb:42:fd:17:28:bf:0d:9b:c2:4f:c0:f6:
  36:6f:86:47:b6:e2:c9:c3:31:2d:6a:50:a2:0f:0f:8b:1d:93:
  4e:3a:ce:c6:6b:5c:cc:48:50:2e:4f:f6:3f:c2:78:37:e5:5a:
  0f:b2:a0:97
```

```
(.python_env) [admin@parrot]~  
└─$ curl -X POST "http://127.0.0.1:8447/validate" \  
-H "accept: application/json" -H "Content-Type: application/json" \  
-d "{  
  \"expiry_date\": \"Feb 19 10:05:53 2026 GMT\",  
  \"issued_date\": \"Dec 8 10:05:53 2025 GMT\",  
  \"issuer\": \"GlobalSign Authority\",  
  \"key_algorithm\": \"rsa\",  
  \"key_size\": 2048,  
  \"key_usage\": [ \"Digital Signature\", \"Key Encipherment\" ], \"primary_domain\": \"api594.pl\",  
  \"san_domains\": [ \"api594.pl\", \"web113.edu\" ]}"  
{  
  "percentage": "100%",  
  "results": {  
    "dates": {  
      "correct": true,  
      "your_expiry": "Feb 19 10:05:53 2026 GMT",  
      "your_issued": "Dec 8 10:05:53 2025 GMT"  
    },  
    "issuer": {  
      "correct": true,  
      "expected": "GlobalSign Authority",  
      "your_answer": "GlobalSign Authority"  
    },  
    "key_info": {  
      "correct": true,  
      "your_algorithm": "RSA",  
      "your_key_size": "2048"  
    },  
    "key_usage": {  
      "correct": true,  
      "expected": [  
        "digital signature",  
        "key encipherment"  
      ]  
    }  
  }  
}
```

```
    ],  
    "your_answer": [  
      "digital signature",  
      "key encipherment"  
    ]  
  },  
  "primary_domain": {  
    "correct": true,  
    "expected": "api594.pl",  
    "your_answer": "api594.pl"  
  },  
  "san_domains": {  
    "correct": true,  
    "expected": [  
      "api594.pl",  
      "web113.edu"  
    ],  
    "your_answer": [  
      "api594.pl",  
      "web113.edu"  
    ]  
  }  
},  
"score": "6/6"  
}
```

6.6 Deszyfrowanie ruchu TLS przy użyciu klucza prywatnego serwera jest możliwe tylko wtedy, gdy połączenie wykorzystuje klasyczną wymianę kluczy RSA (tzw. `TLS_RSA`). W takim przypadku klient szyfruje pre-master secret kluczem publicznym serwera, a posiadanie klucza prywatnego pozwala ten sekret odszyfrować. Po jego odzyskaniu można wyliczyć master secret oraz klucze sesyjne, co umożliwia odtworzenie pełnej, zaszyfrowanej sesji TLS. W praktyce oznacza to, że mając klucz prywatny, da się odszyfrować ruch wszystkich klientów, którzy łączyli się z danym serwerem za pomocą RSA-based key exchange, o ile dysponujemy przechwyconym ruchem. Nie działa to jednak w przypadku nowoczesnych połączeń używających ECDHE lub DHE, gdzie wykorzystywana jest tzw. forward secrecy. Tam klucz prywatny serwera nie wystarcza do deszyfrowania sesji, ponieważ każdy klient i serwer uzgadniają tymczasowe, jednorazowe klucze, które nie są nigdzie zapisane i których nie można odtworzyć z samego klucza prywatnego.

- (a) Twoim zadaniem jest odszyfrowanie ruchu sieciowego posiadając klucz prywatny serwera.
- (b) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 8445:443 --name tls4 docker.io/mazurkatarzyna/tls-ex4:latest
podman run -p 8445:443 --name tls4 docker.io/mazurkatarzyna/tls-ex4:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 8447:443 --name tls4 ghcr.io/mazurkatarzyna/tls-ex4:latest
podman run -p 8447:443 --name tls4 ghcr.io/mazurkatarzyna/tls-ex4:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <https://127.0.0.1:8447>.

- (c) Otwórz w Wireshark'u plik `nettraffic1.pcapng`. Czy jesteś w stanie odczytać jego zawartość?
- (d) Dodaj do Wireshark'a klucz prywatny serwera:

```
Edit → Preferences → Protocols → TLS → RSA keys list
```

z wartościami:

```
0.0.0.0, 8443, http, /ścieżka/do/server.key
```

- (e) Sprawdź, czy ruch sieciowy został odszyfrowany.
- (f) Wykorzystaj serwer działający pod adresem <https://127.0.0.1:8447> i za pomocą Wireshark'a przechwycić ruch sieciowy do serwera. Następnie odszyfruj komunikację przy użyciu klucza prywatnego serwera.